Antigravity Editor: MCP Integration

Antigravity supports the Model Context Protocol (MCP), a standard that allows the editor to securely connect to your local tools, databases, and external services. This integration provides the AI with real-time context beyond just the files open in your editor.

What is MCP?

MCP acts as a bridge between Antigravity and your broader development environment. Instead of manually pasting context (like database schemas or logs) into the editor, MCP allows Antigravity to fetch this information directly when needed.

Core Features

1. Context Resources

The AI can read data from connected MCP servers to inform its suggestions.

Example: When writing a SQL query, Antigravity can inspect your live Neon or Supabase schema to suggest correct table and column names.

Example: When debugging, the editor can pull in recent build logs from Netlify or Heroku.

2. Custom Tools

MCP enables Antigravity to execute specific, safe actions defined by your connected servers.

Example: "Create a Linear issue for this TODO."

Example: "Search Notion or GitHub for authentication patterns."

How to Connect

Connections are managed directly through the built-in MCP Store.

1. **Access the Store:** Open the MCP Store panel within the "..." dropdown at the top of the editor's side panel.
2. **Browse & Install:** Select any of the supported servers from the list and click Install.
3. **Authenticate:** Follow the on-screen prompts to securely link your accounts (where applicable).

To connect to a custom MCP server:

1. Open the MCP store via the "..." dropdown at the top of the editor's agent panel.
2. Click on "Manage MCP Servers"
3. Click on "View raw config"
4. Modify the mcp_config.json with your custom MCP server configuration.

The configuration file is located at `~/.gemini/antigravity/mcp_config.json`.

Configuration Structure

The configuration file has a single `mcpServers` object where you define each server you want to connect to.

json

```
{
  "mcpServers": {
    "serverName": {
      "command": "path/to/executable",
      "args": [
        "--arg1",
        "value1"
      ],
      "env": {
        "API_KEY": "your-api-key"
      }
    }
  }
}
```

Configuration Properties

Each server entry supports the following properties:

Transport (one required):

- **`command`** (string): Path to the executable for stdio transport.

- `args` (string[]): Command-line arguments for stdio transport.
- `env` (object): Environment variables for the stdio server process.
- `cwd` (string): Working directory for stdio servers.
- `headers` (object): Custom HTTP headers for remote servers.
- `authProviderType` (string): Authentication provider. Supports `"google_credentials"` for ADC.
- `oauth` (object): OAuth client credentials (`clientId` , `clientSecret`).
- `disabled` (boolean): Temporarily disable a server without removing its configuration.
- `disabledTools` (string[]): Tool names to not provide to the model.

Authentication

Google Credentials

Set `authProviderType` to `"google_credentials"` to use Google Application Default Credentials (ADC).

json

```
{
  "mcpServers": {
    "my-gcp-service": {
      "serverUrl":
"https://example.googleapis.com/mcp/",
      "authProviderType": "google_credentials"
    }
  }
}
```

This requires Application Default Credentials to be configured. To set them up, run:

bash

```
gcloud auth application-default login
```

needed:

```json
{
  "mcpServers": {
    "oauth-server": {
      "serverUrl": "https://api.example.com/mcp/"
    }
  }
}
```

If the server does not support dynamic client registration, you can provide your client credentials manually:

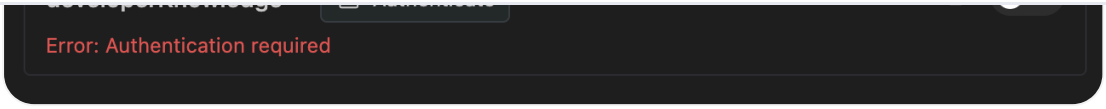
```json
{
  "mcpServers": {
    "oauth-server": {
      "serverUrl": "https://api.example.com/mcp/",
      "oauth": {
        "clientId": "your-client-id",
        "clientSecret": "your-client-secret"
      }
    }
  }
}
```

If you provided client credentials manually, ensure the following is registered as a redirect URI in your OAuth provider:

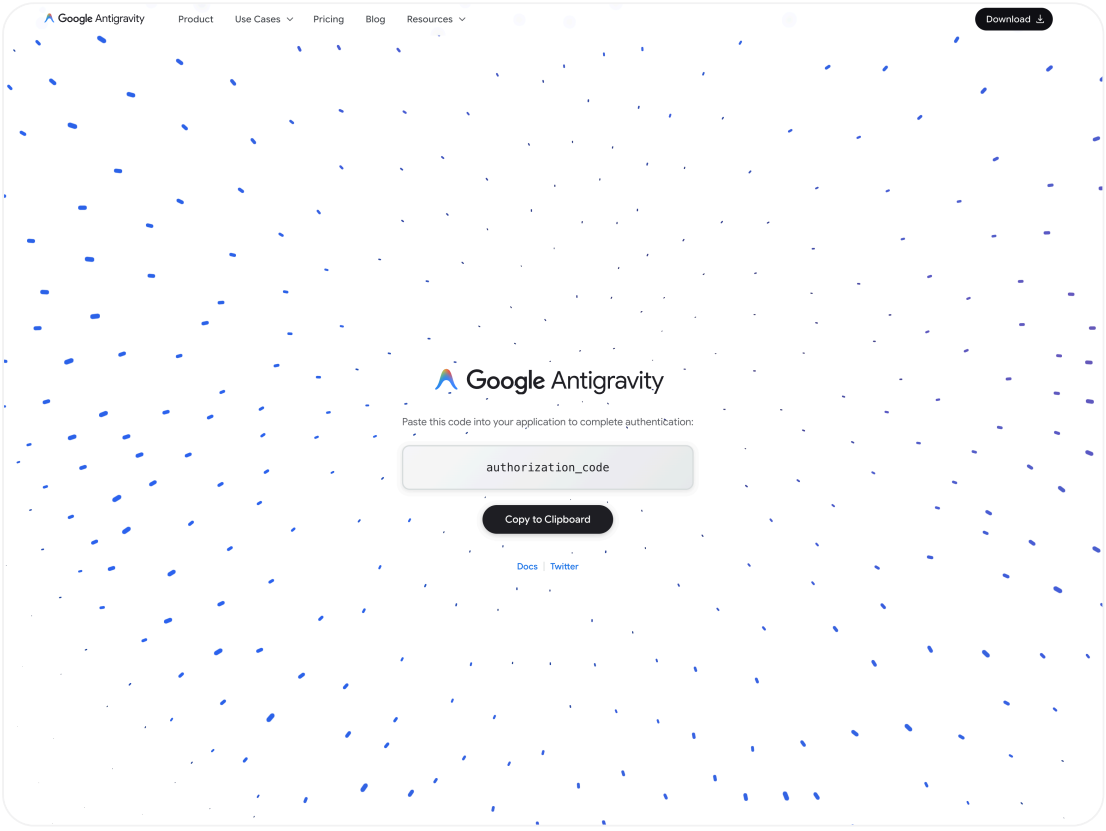
```
https://antigravity.google/oauth-callback
```

When connecting to an OAuth-enabled server:

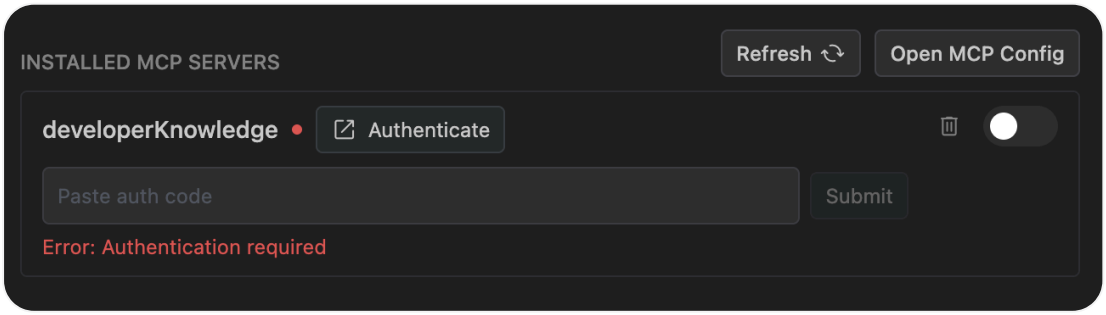
1. Open **Agent Settings** with `Cmd+,` (Mac) or `Ctrl+,` (Windows/Linux).
2. Navigate to the **Customizations** tab and click the **Authenticate** button next to the server.



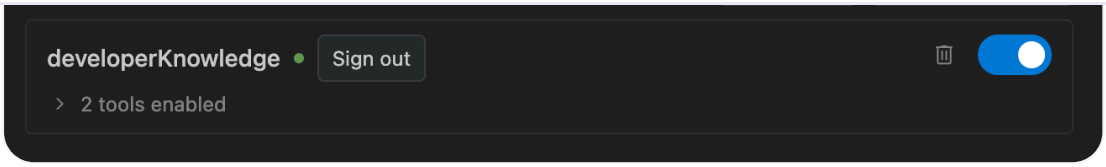
1. Complete authentication in your browser and copy the authorization code.



1. Paste the code back into the settings panel and click **Submit**.



Tools > MCP



Access tokens are stored in `~/.gemini/antigravity/mcp_oauth_tokens.json`. Expired tokens are refreshed automatically, and invalid tokens are removed.

Custom Headers

For servers that require custom HTTP headers (e.g. API keys or bearer tokens), add them to the `headers` object. For example:

json

```
{
  "mcpServers": {
    "my-remote-server": {
      "serverUrl": "https://api.example.com/mcp/",
      "headers": {
        "Authorization": "Bearer YOUR_API_TOKEN"
      }
    }
  }
}
```

Supported Servers

The MCP Store currently features integrations for:

- Airweave
- Arize
- AlloyDB for PostgreSQL
- Atlassian
- BigQuery
- Chrome DevTools
- ClickHouse
- Cloud SQL for PostgreSQL

- ~~Dataplex~~
- Figma Dev Mode MCP
- Firebase
- GitHub
- Harness
- Heroku
- Linear
- Locofy
- Looker
- MCP Toolbox for Databases
- MongoDB
- Neon
- Netlify
- Notion
- PayPal
- Perplexity Ask
- Pinecone
- Postman
- Prisma
- Redis
- Sequential Thinking
- SonarQube
- Spanner
- Stripe
- Supabase

‹ Sandboxing Terminal Commands

Artifacts ›